```
import numpy as np
import seaborn as sns


y = lambda x: (1 + x) * np.exp(x * x)

start, end = 0, 1
eps = 1e-3
h = 0.1

x = np.arange(start, end + h, h)
```

$$\begin{cases} y'' - 4xy' + (4x^2 - 2)y = 0, \\ y(0) = 1, \\ y'(0) = 1, \\ x \in [0, 1], \\ h = 0.1, \end{cases} \qquad y = (1 + x)e^{x^2}$$

Метод Эйлера второго порядка точности

Преобразуем дифференциальное уравнение второго порядка к системе двух уравнений первого порядка, вводя $z = y'$:

$$y' = z, \qquad z' = 4xz - (4x^2 - 2)y.$$

Также посчитаем значения решения на заданной сетке:

```
f_der_1 = lambda z: z
f_der_2 = lambda x, y, z: 4 * x * z - (4 * x * x - 2) * y

y_etalon_f = y(x)
```

Определим функцию, которая считает приближённое решение системы методом Эйлера второго порядка точности для начальных условий $y(x_0) = y_0$ и $z(x_0) = z_0$ с фиксированным шагом h:

```
def euler(y_der_1, y_der_2, x, y0, z0, h):
    y = np.zeros(len(x), dtype=np.float64)
    z = np.zeros(len(x), dtype=np.float64)
    y[0], z[0] = y0, z0

    for i in range(1, len(x)):
        x_k_1_2 = x[i - 1] + h / 2
        y_k_1_2 = y[i - 1] + h / 2 * y_der_1(z[i - 1])
        z_k_1_2 = z[i - 1] + h / 2 * y_der_2(x[i - 1], y[i - 1], z[i - 1])

        y[i] = y[i - 1] + h * y_der_1(z_k_1_2)
        z[i] = z[i - 1] + h * y_der_2(x_k_1_2, y_k_1_2, z_k_1_2)
```

```
    return y, z
```

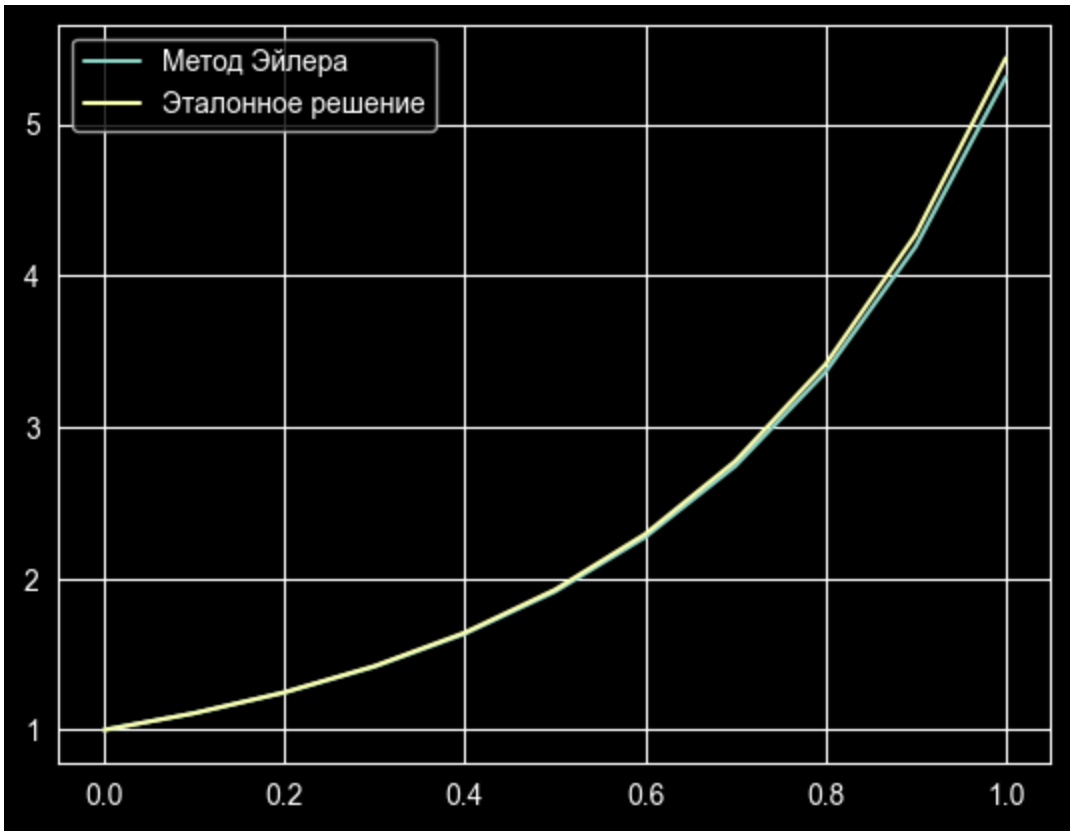
Посчитаем приближённые значения y и z методом Эйлера второго порядка точности на отрезке $[start, end]$ с шагом h:

```
y_euler, z_euler = euler(f_der_1, f_der_2, x, 1, 1, h)
```

Построим графики приближённого решения, полученного методом Эйлера, и эталонного решения и проверим корректность реализованного метода:

```
sns.lineplot(x=x, y=y_euler, label='Метод Эйлера')
sns.lineplot(x=x, y=y_etalon_f, label='Эталонное решение')
```

```
<Axes: >
```



Метод Рунге-Кутта четвёртого порядка точности

Определим функцию, которая считает приближённое решение системы методом Рунге-Кутта четвёртого порядка точности для начальных условий $y(x_0) = y_0$ и $z(x_0) = z_0$ с фиксированным шагом h:

```
def runge_kutta(y_der_1, y_der_2, x, y0, z0, h):
    y = np.zeros(len(x), dtype=np.float64)
    z = np.zeros(len(x), dtype=np.float64)

    y[0], z[0] = y0, z0
```

```
    for i in range(1, len(x)):
        K_k_1 = h * y_der_1(z[i - 1])
        L_k_1 = h * y_der_2(x[i - 1], y[i - 1], z[i - 1])


        K_k_2 = h * y_der_1(z[i - 1] + L_k_1 / 2)

        L_k_2 = h * y_der_2(
            x[i - 1] + h / 2,
            y[i - 1] + K_k_1 / 2,
            z[i - 1] + L_k_1 / 2
        )


        K_k_3 = h * y_der_1(z[i - 1] + L_k_2 / 2)

        L_k_3 = h * y_der_2(
            x[i - 1] + h / 2,
            y[i - 1] + K_k_2 / 2,
            z[i - 1] + L_k_2 / 2
        )


        K_k_4 = h * y_der_1(z[i - 1] + L_k_3)

        L_k_4 = h * y_der_2(
            x[i - 1] + h,
            y[i - 1] + K_k_3,
            z[i - 1] + L_k_3
        )


        y_delta = (K_k_1 + 2 * K_k_2 + 2 * K_k_3 + K_k_4) / 6
        z_delta = (L_k_1 + 2 * L_k_2 + 2 * L_k_3 + L_k_4) / 6


        y[i] = y[i - 1] + y_delta
        z[i] = z[i - 1] + z_delta

    return y, z
```

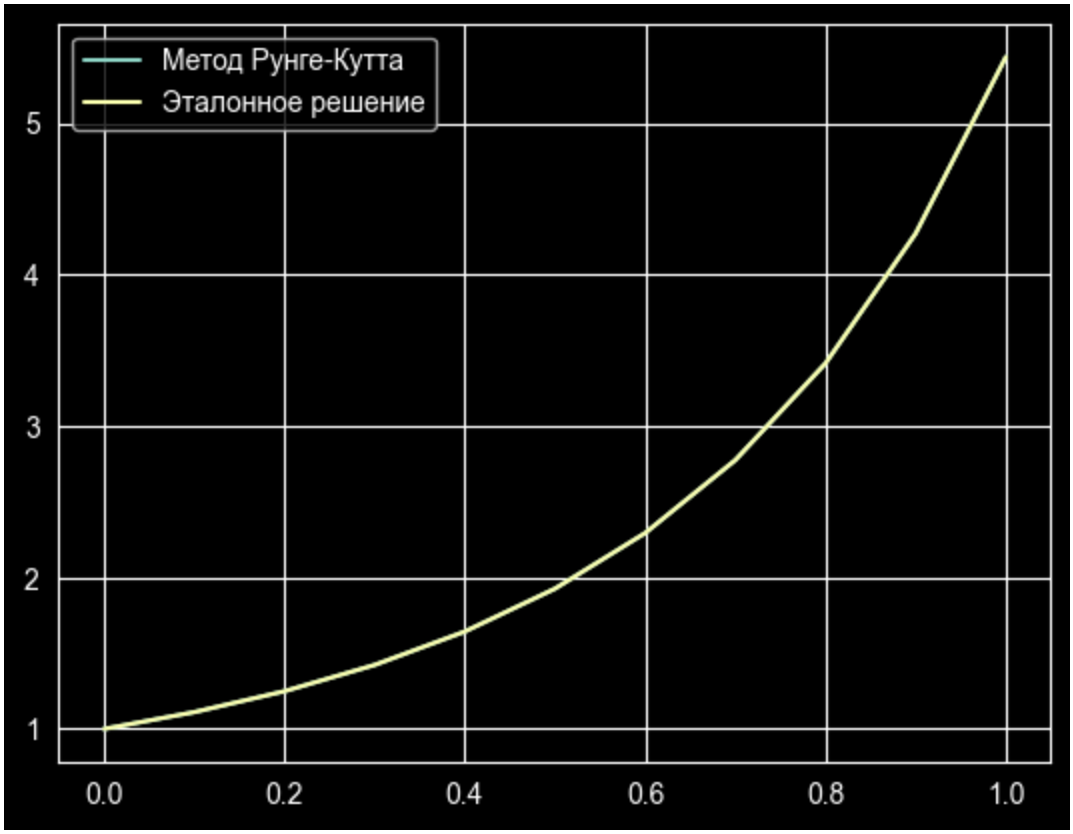
Посчитаем приближённые значения y и z методом Рунге-Кутта четвёртого порядка точности на отрезке $[start, end]$ с шагом h:

```
y_runge_kutta, z_runge_kutta = runge_kutta(f_der_1, f_der_2, x, 1, 1, h)
```

Построим графики приближённого решения, полученного методом Рунге-Кутта, и эталонного решения. Проверим корректность реализованного метода:

```
sns.lineplot(x=x, y=y_runge_kutta, label='Метод Рунге-Кутта')
sns.lineplot(x=x, y=y_etalon_f, label='Эталонное решение')
```

```
<Axes: >
```



Метод Адамса четвёртого порядка точности

Определим функцию, которая считает приближённое решение системы явным методом Адамса четвёртого порядка точности. Первые четыре значения берутся из уже найденного решения методом Рунге-Кутта:

```
def adams(y_der_1, y_der_2, x, y_start, z_start, h):
    y = np.concat((y_start, np.zeros(len(x) - 4, dtype=np.float64)))
    z = np.concat((z_start, np.zeros(len(x) - 4, dtype=np.float64)))


    for i in range(4, len(x)):
        y[i] = y[i - 1] + (h / 24) * (
            55 * y_der_1(z[i - 1])
            - 59 * y_der_1(z[i - 2])
            + 37 * y_der_1(z[i - 3])
            - 9 * y_der_1(z[i - 4])
        )

        z[i] = z[i - 1] + (h / 24) * (
            55 * y_der_2(x[i - 1], y[i - 1], z[i - 1])
            - 59 * y_der_2(x[i - 2], y[i - 2], z[i - 2])
            + 37 * y_der_2(x[i - 3], y[i - 3], z[i - 3])
            - 9 * y_der_2(x[i - 4], y[i - 4], z[i - 4])
        )

    return y, z
```

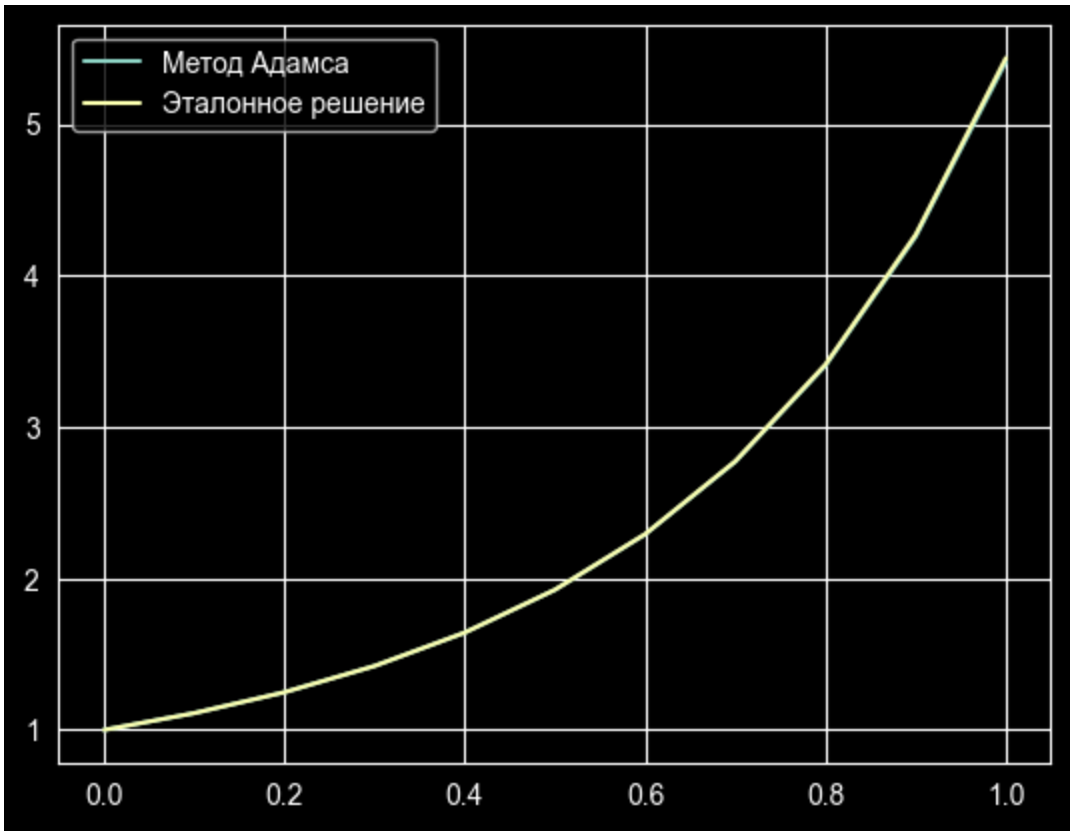
Посчитаем приближённые значения y и z методом Адамса четвёртого порядка точности, используя первые четыре точки решения, полученного методом Рунге-Кутта:

```
y_adams, z_adams = adams(
    f_der_1,
    f_der_2,
    x,
    y_runge_kutta[:4],
    z_runge_kutta[:4],
    h
)
```

Построим графики приближённого решения, полученного методом Адамса, и эталонного решения. Проверим корректность реализованного метода:

```
sns.lineplot(x=x, y=y_adams, label='Метод Адамса')
sns.lineplot(x=x, y=y_etalon_f, label='Эталонное решение')
```

```
<Axes: >
```



Метод Рунге-Ромберга-Ричардсона для контроля точности

Определим функцию, которая считает приближённое значение погрешности по формуле метода Рунге-Ромберга-Ричардсона для решений, полученных с шагами h и kh, при порядке точности p:

```
def runge_romberg_richardson(f_h, f_kh, p, k_coef):
    return (f_h[::k_coef] - f_kh) / (k_coef ** p - 1)
```

Уменьшим шаг в k раз, построим новую сетку и повторно найдём приближённые решения методами Эйлера, Рунге-Кутта и Адамса:

```
k = 2
smaller_h = h / k
x_smaller_h = np.arange(start, end + smaller_h, smaller_h)

y_euler_smaller_h, z_euler_smaller_h = euler(
    f_der_1, f_der_2, x_smaller_h,
    1, 1, smaller_h
)
y_runge_kutta_smaller_h, z_runge_kutta_smaller_h = runge_kutta(
    f_der_1, f_der_2, x_smaller_h,
    1, 1, smaller_h
)
y_adams_smaller_h, z_adams_smaller_h = adams(
    f_der_1, f_der_2, x_smaller_h,
    y_runge_kutta_smaller_h[:4], z_runge_kutta_smaller_h[:4], smaller_h
)
```

Найдём приближённые значения погрешности по формуле метода Рунге-Ромберга-Ричардсона для методов Эйлера, Рунге-Кутта и Адамса:

```
euler_runge_romberg = runge_romberg_richardson(
    y_euler_smaller_h,
    y_euler, 2, k
)
runge_kutta_runge_romberg = runge_romberg_richardson(
    y_runge_kutta_smaller_h,
    y_runge_kutta, 4, k
)
adams_runge_romberg = runge_romberg_richardson(
    y_adams_smaller_h,
    y_adams, 4, k
)
```

Определим функцию, которая считает суммарную абсолютную погрешность приближённого решения относительно эталонного решения:

```
def error(v, v_etalon):
    return np.sum(np.abs(v - v_etalon))
```

Уточним приближённые решения, добавив к ним оценки погрешностей, полученные по формуле Рунге-Ромберга-Ричардсона:

```
y_euler_runge_romberg = y_euler + euler_runge_romberg
y_runge_kutta_runge_romberg = y_runge_kutta + runge_kutta_runge_romberg
y_adams_runge_romberg = y_adams + adams_runge_romberg
```

Валидация

Проверим максимальные по модулю оценки погрешности для методов Эйлера, Рунге-Кутта и Адамса, чтобы убедиться в корректности их подсчёта:

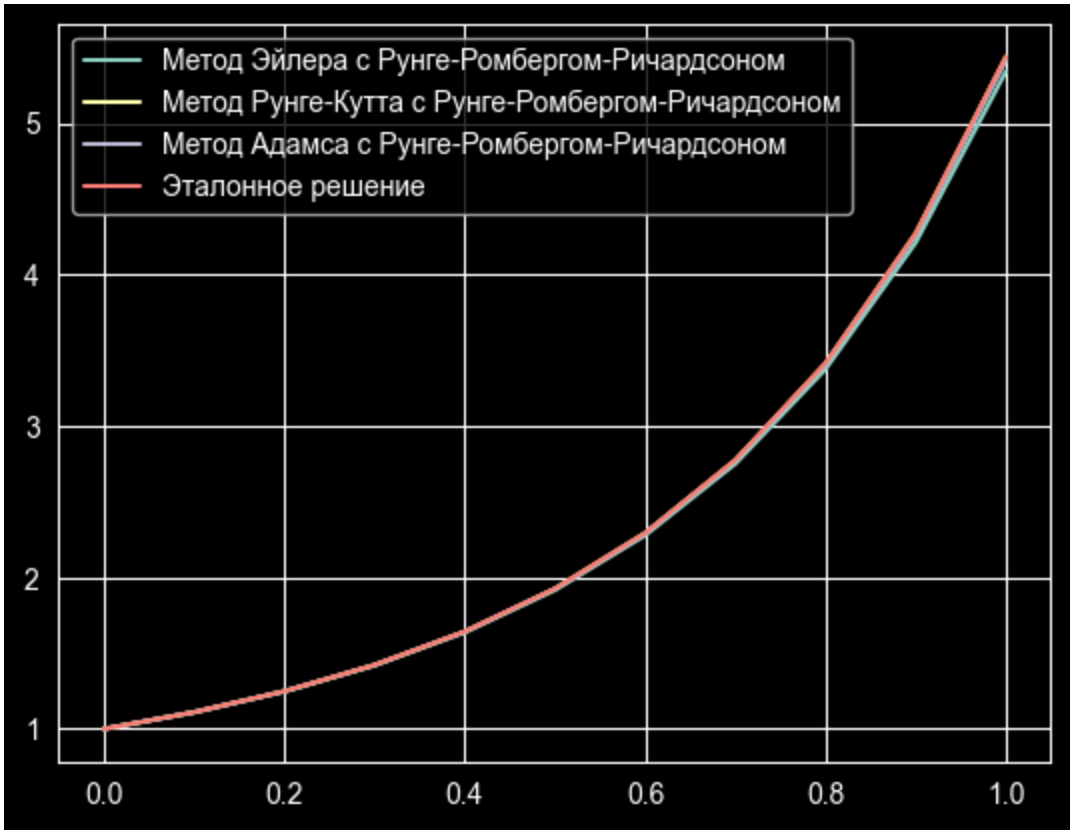
```
print(
    np.max(np.abs(euler_runge_romberg)),
    np.max(np.abs(runge_kutta_runge_romberg)),
    np.max(np.abs(adams_runge_romberg)),
)
```

```
0.030417192786919678 2.568946429786223e-05 0.001526943109977689
```

Построим графики уточнённых решений после применения метода Рунге-Ромберга-Ричардсона и сравним их с эталонным решением:

```
sns.lineplot(
    x=x, y=y_euler_runge_romberg,
    label='Метод Эйлера с Рунге-Ромбергом-Ричардсоном'
)
sns.lineplot(
    x=x, y=y_runge_kutta_runge_romberg,
    label='Метод Рунге-Кутта с Рунге-Ромбергом-Ричардсоном'
)
sns.lineplot(
    x=x, y=y_adams_runge_romberg,
    label='Метод Адамса с Рунге-Ромбергом-Ричардсоном'
)
sns.lineplot(
    x=x, y=y_etalon_f,
    label='Эталонное решение'
)
```

```
<Axes: >
```



Посчитаем суммарные абсолютные погрешности уточнённых решений относительно эталонного решения:

```
print(
    "Погрешность метода Эйлера с Рунге-Ромбергом-Ричардсоном:",
    error(y_euler_runge_romberg, y_etalon_f)
)
print(
    "Погрешность метода Рунге-Кутта с Рунге-Ромбергом-Ричардсоном:",
    error(y_runge_kutta_runge_romberg, y_etalon_f)
)
print(
    "Погрешность метода Адамса с Рунге-Ромбергом-Ричардсоном:",
    error(y_adams_runge_romberg, y_etalon_f)
)
```

```
Погрешность метода Эйлера с Рунге-Ромбергом-Ричардсоном: 0.25910783314719277
Погрешность метода Рунге-Кутта с Рунге-Ромбергом-Ричардсоном: 0.0009050781446588996
Погрешность метода Адамса с Рунге-Ромбергом-Ричардсоном: 0.053685829495393245
```